1. Что такое переменные и какие способы их объявления существуют?

Переменная — это «контейнер» для хранения данных: числа, строки, объектов и т.д. В JavaScript переменные позволяют сохранять значения и потом использовать их по имени в разных местах программы.

Основные способы объявления:

- `let` Объявляет переменную с блочной областью видимости (действует внутри ближайших `{}`), значение можно менять.

  js

  ```
  let age = 25;
  age = 26;
  ```

- `const` Тоже блочная область видимости, но **нельзя** переопределить саму переменную (нельзя сделать `age = 30`), однако можно изменять внутреннее состояние объекта или массива.

  js

  ```
  const user = { name: 'Alex' };
  user.name = 'Max'; // можно
  // user = {} // нельзя
  ```

- `var` Старый способ. Имеет функциональную или глобальную область видимости, подвержен «всплытию» (hoisting) и ведет себя менее предсказуемо, поэтому в современном коде почти не используется.

  js

  ```
  var x = 10;
  ```

2. В чём отличие `var`, `let` и `const`?

Ключевые отличия:

- **Область видимости:**
  - `var` — функциональная или глобальная.
  - `let` **и** `const` — блочная (внутри `{}`).
- **Переприсваивание и переобъявление:**
  - `var` — можно и переприсвоить, и переобъявить в той же области.
  - `let` — можно переприсвоить, но нельзя переобъявить в той же области.
  - `const` — нельзя ни переприсвоить, ни переобъявить.
- **Hoisting (всплытие):**
  - Все три всплывают, но:
    - `var` при обращении до строки объявления даёт `undefined`.
    - `let` и `const` до фактической инициализации находятся в «temporal dead zone» и вызовут ошибку.

На практике: **используют** `const` **по умолчанию**, а `let` — когда значение действительно нужно менять. `var` — избегают.

3. Что такое область видимости?

Область видимости (scope) — это «зона», в пределах которой переменная доступна по имени.

Основные виды:

- **Глобальная область** — переменные доступны везде.
- **Функциональная область** — переменные, объявленные внутри функции (`var`), доступны только внутри этой функции.
- **Блочная область** — переменные, объявленные через `let` и `const` внутри `{}`, доступны только внутри этого блока:

  js

  ```
  if (true) {
    let a = 10;
  }
  // a здесь недоступна
  ```

Область видимости помогает изолировать данные и избегать конфликтов имен.

4. Какие существуют типы данных в JS?

В JavaScript есть примитивные и ссылочные типы.

Примитивные:

- `number` — числа (целые и с плавающей точкой).
- `string` — строки.
- `boolean` — `true` или `false`.
- `null` — «пустое» значение.
- `undefined` — значение не было присвоено.
- `symbol` — уникальные идентификаторы.
- `bigint` — большие целые числа.

Ссылочный:

- `object` — объекты, массивы, функции и т.д.

5. Разница между `==` и `===`

- `==` **(нестрогое сравнение)** Сравнивает значения **с приведением типов**.

  js

  ```
  5 == '5' // true
  ```

- `===` **(строгое сравнение)** Сравнивает и тип, и значение **без приведения типов**.

js

```
5 === '5' // false
5 === 5   // true
```

В современном коде почти всегда используют `===`, чтобы избежать неожиданных преобразований.

6. Что такое `NaN` и как его проверить?

`NaN` (Not-a-Number) — специальное значение типа `number`, которое означает «не число», обычно результат некорректной математической операции:

js
```
const x = 0 / 0; // NaN
```

Проверка:

- `Number.isNaN(value)` — корректный способ:

  js

  ```
  Number.isNaN(NaN); // true
  Number.isNaN('text'); // false
  ```

- Глобальная функция `isNaN` менее точна, потому что сначала приводит к числу.

7. Что такое шаблонные строки?

Шаблонные строки (template literals) — это строки в обратных кавычках `` `, которые позволяют:

- Вставлять выражения через `${}`:

  js

  ```
  const name = 'Alex';
  const msg = `Привет, ${name}!`;
  ```

- Писать многострочные строки без `\n`:

  js

  ```
  const text = `Первая строка
  Вторая строка`;
  ```

Они делают работу со строками намного удобнее.

8. Что возвращает оператор `typeof`?

`typeof` возвращает строку с названием типа значения:

js

```
typeof 5;           // "number"
typeof 'hello';     // "string"
typeof true;        // "boolean"
typeof undefined;   // "undefined"
typeof null;        // "object" (исторический баг)
typeof {};          // "object"
typeof [];          // "object"
typeof function(){}; // "function"
```

Важно помнить: для `null` результат `"object"` — это особенность языка.

9. Разница между function declaration и function expression

- **Function Declaration** — объявление функции отдельной конструкцией:

  js

  ```
  function sum(a, b) {
    return a + b;
  }
  ```

  Особенность: такие функции **всплывают** — их можно вызывать до места объявления.

- **Function Expression** — функция, записанная в переменную:

  js

  ```
  const sum = function(a, b) {
    return a + b;
  };
  ```

  Здесь функция создается только в момент выполнения этой строки, до этого использовать нельзя.

Также есть **стрелочные функции**:

js
```
const sum = (a, b) => a + b;
```

10. Что такое объект и как его создать?

Объект — это набор пар «ключ–значение». Используется для хранения структурированных данных.

Способы создания:

- **Литерал объекта:**

  js

  ```
  const user = {
    name: 'Alex',
    age: 25
  };
  ```

- **Через** `new Object()`**:**

  js

  ```
  const user = new Object();
  user.name = 'Alex';
  ```

- **Через класс или функцию-конструктор:**

  js

  ```
  function User(name) {
    this.name = name;
  }
  const u = new User('Alex');
  ```

11. Что делает метод `toUpperCase` и `toLowerCase`?

Оба метода работают со строками:

- `toUpperCase()` — возвращает новую строку, где все буквы в верхнем регистре:

  js

  ```
  'hello'.toUpperCase(); // "HELLO"
  ```

- `toLowerCase()` — возвращает строку в нижнем регистре:

  js

  ```
  'HeLLo'.toLowerCase(); // "hello"
  ```

Исходная строка не изменяется (строки — неизменяемы).

12. Что такое классы в JS?

Классы — это синтаксический сахар над прототипным наследованием. Они позволяют описывать «шаблон» для объектов:

js
```
class User {
  constructor(name) {
    this.name = name;
  }

  sayHi() {
    console.log(`Привет, я ${this.name}`);
  }
}

const u = new User('Alex');
u.sayHi();
```

Класс может иметь:

- **конструктор** (`constructor`)
- **методы**
- **наследование** через `extends`.

13. Что такое цикл `for` и как он используется?

Цикл `for` — конструкция для многократного выполнения кода.

Общий вид:

js
```
for (инициализация; условие; шаг) {
  // тело цикла
}
```

Пример:

js
```
for (let i = 0; i < 5; i++) {
  console.log(i);
}
```

- `let i = 0` — стартовое значение.
- `i < 5` — условие продолжения.
- `i++` — изменение на каждом шаге.

14. Что такое методы `push` и `pop` у массива?

Оба метода изменяют массив **с конца**:

- `push(value)` — добавляет элемент в конец массива и возвращает новую длину:

  js
  ```
  const arr = [1, 2];
  arr.push(3); // arr = [1, 2, 3]
  ```

- `pop()` — удаляет последний элемент и возвращает его:

  js
  ```
  const arr = [1, 2, 3];
  const last = arr.pop(); // last = 3, arr = [1, 2]
  ```

15. Разница между синхронным и асинхронным кодом

- **Синхронный код** выполняется по порядку, строка за строкой. Пока одна операция не завершится, следующая не начнется.
- **Асинхронный код** позволяет не блокировать выполнение программы во время долгих операций (запросы к серверу, таймеры). Вместо этого регистрируются колбэки, промисы или `async/await`, которые срабатывают позже, когда результат готов.

Асинхронность в JS реализуется через **event loop** и очередь задач.

16. Что такое промисы?

Промис — это объект, представляющий результат асинхронной операции: **успех** или **ошибка**.

Состояния промиса:

- `pending` — в ожидании.
- `fulfilled` — успешно выполнен.
- `rejected` — завершился с ошибкой.

Пример:

js
```
const promise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve('Готово');
  }, 1000);
});

promise
  .then(result => console.log(result))
  .catch(error => console.error(error));
```

С промисами удобно использовать `async/await`.

17. Как работают таймеры (`setTimeout`, `setInterval`)?

- `setTimeout(fn, delay)` — выполняет функцию **один раз** через указанное время (в миллисекундах):

  js
  ```
  setTimeout(() => {
    console.log('Через 1 секунду');
  }, 1000);
  ```

- `setInterval(fn, delay)` — выполняет функцию **периодически**, каждые `delay` миллисекунд:

  js
  ```
  const id = setInterval(() => {
    console.log('Каждую секунду');
  }, 1000);
  ```

Оба возвращают ID таймера, который можно остановить через `clearTimeout(id)` или `clearInterval(id)`.

18. Что такое модули и как их импортировать/экспортировать?

Модули — это способ разбить код на файлы и управлять зависимостями.

Экспорт:

- **Именованный экспорт:**

  js

  ```
  export const a = 10;
  export function sum() {}
  ```

- **Экспорт по умолчанию:**

  js

  ```
  export default function main() {}
  ```

Импорт:

- Именованный:

  js

  ```
  import { a, sum } from './module.js';
  ```

- По умолчанию:

  js

  ```
  import main from './module.js';
  ```

Можно комбинировать:

js
```
import main, { a, sum } from './module.js';
```

19. Что такое React и зачем он нужен?

React — это библиотека JavaScript для построения пользовательских интерфейсов. Она помогает:

- разбивать интерфейс на **компоненты**;
- эффективно обновлять только те части DOM, которые изменились (через виртуальный DOM);
- управлять состоянием приложения.

React упрощает создание сложных, динамических интерфейсов, особенно когда много взаимодействий и состояний.

20. Что такое компоненты в React?

Компонент — это «кусочек» интерфейса, который:

- имеет свою разметку (JSX),
- может принимать данные через **props**,

- может иметь внутреннее **state** (для функциональных — через хуки).

Примеры:

- **Функциональный компонент:**

  jsx

  ```
  function Button(props) {
    return <button>{props.text}</button>;
  }
  ```

- **Стрелочный:**

  jsx

  ```
  const Button = ({ text }) => <button>{text}</button>;
  ```

Компоненты можно переиспользовать и комбинировать.

21. Что такое JSX?

JSX — это синтаксис, похожий на HTML, который пишется прямо в JavaScript и затем компилируется в вызовы `React.createElement`.

Пример:

jsx
```
const element = <h1>Привет, мир!</h1>;
```

Особенности:

- Внутри JSX можно вставлять JS-выражения в `{}`:

  jsx

  ```
  const name = 'Alex';
  const element = <h1>Привет, {name}</h1>;
  ```

- Атрибуты пишутся в `camelCase` (`className`, `onClick` и т.д.).

22. Что такое пропсы (props)?

Props — это «свойства» компонента, данные, которые передаются **снаружи**.

Пример передачи:

jsx
```
function Greeting(props) {
  return <h1>Привет, {props.name}</h1>;
}

<Greeting name="Alex" />;
```

Особенности:

- Props **только для чтения** внутри компонента.
- Компонент не должен изменять свои props, а только использовать их.

23. Что такое состояние (state)?

State — это внутренние данные компонента, которые могут меняться со временем и влияют на то, что отображается.

В функциональных компонентах состояние задается через хук `useState`:

jsx
```
const [count, setCount] = useState(0);
```

Когда состояние меняется через `setCount`, React повторно рендерит компонент с новым значением.

24. Как обрабатывать события в React?

События в React похожи на DOM-события, но:

- пишутся в `camelCase` (`onClick`, `onChange`),
- в качестве обработчика передается **функция**, а не строка.

Пример:

jsx
```
function Button() {
  function handleClick() {
    console.log('Нажали');
  }

  return <button onClick={handleClick}>Нажми меня</button>;
}
```

Можно использовать стрелочные функции:

jsx
```
<button onClick={() => console.log('Нажали')}>Нажми</button>
```

25. Что такое хуки (hooks)?

Хуки — это функции, которые позволяют функциональным компонентам использовать:

- состояние (`useState`),
- эффекты (`useEffect`),
- контекст (`useContext`),
- и другие возможности React.

Пример:

jsx

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

Правила:

- вызывать хуки только **на верхнем уровне** компонента;
- вызывать только внутри **функциональных компонентов** или своих хуков.

26. Как работает `useState`?

`useState` — хук для управления состоянием в функциональном компоненте.

Сигнатура:

jsx
```
const [value, setValue] = useState(initialValue);
```

- `value` — текущее значение состояния.
- `setValue` — функция, которая обновляет состояние и запускает повторный рендер.

Пример:

jsx
```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Счетчик: {count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
    </div>
  );
}
```

27. Что такое React Router?

React Router — библиотека для организации маршрутизации (страниц) в React-приложении.

Позволяет:

- отображать разные компоненты в зависимости от URL;
- создавать ссылки (`<Link>`), переходы (`useNavigate`);
- определять маршруты (`<Route>` внутри `<Routes>`).

Пример (v6):

jsx
```
import { BrowserRouter, Routes, Route } from 'react-router-dom';

function App() {
```

```
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

28. Как отобразить список элементов в React?

Обычно используют метод массива `map`:

jsx
```
const items = ['яблоко', 'банан', 'груша'];

function List() {
  return (
    <ul>
      {items.map((item, index) => (
        <li key={index}>{item}</li>
      ))}
    </ul>
  );
}
```

Важно: каждому элементу списка нужно задать **уникальный** `key`, чтобы React корректно отслеживал изменения.

29. Что такое событие `onClick` в React?

`onClick` — это обработчик события клика по элементу.

Пример:

jsx
```
<button onClick={() => console.log('Клик!')}>
  Нажми меня
</button>
```

React сам повесит слушатель на соответствующий DOM-элемент. Главное — передать **функцию**, а не результат её вызова.

30. Как обработать ввод текста в поле (`input`) в React?

Обычно делают **контролируемый компонент**: значение хранится в state, а `onChange` обновляет его.

jsx
```
import { useState } from 'react';

function TextInput() {
  const [value, setValue] = useState('');

  function handleChange(event) {
    setValue(event.target.value);
```

```
  }

  return (
    <input
      type="text"
      value={value}
      onChange={handleChange}
    />
  );
}
```

Так React всегда знает текущее значение поля.

31. Как подключить внешний CSS-файл к компоненту?

Самый простой способ — импортировать CSS-файл в JS-файл компонента:

jsx
```
// Button.jsx
import './Button.css';

function Button() {
  return <button className="btn">Кнопка</button>;
}

export default Button;
```

А в `Button.css`:

css
```
.btn {
  background-color: blue;
  color: white;
}
```

Сборщик (например, Create React App, Vite) обработает этот импорт.

32. Что такое список элементов и как его отобразить с помощью `map`?

Список элементов — это набор однотипных данных (например, массив объектов или строк), который нужно вывести в интерфейсе.

Используем `map`:

jsx
```
const users = [
  { id: 1, name: 'Alex' },
  { id: 2, name: 'Max' },
];

function UsersList() {
  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
```

```
}
```

- `map` возвращает новый массив JSX-элементов.
- `key` должен быть стабильным и уникальным (часто используют `id`).

33. Что такое событие `onChange` и где оно применяется?

`onChange` — событие изменения значения элемента формы.

Чаще всего используется с:

- `<input>`
- `<textarea>`
- `<select>`

Пример:

jsx
```
function Input() {
  const [value, setValue] = useState('');

  return (
    <input
      value={value}
      onChange={e => setValue(e.target.value)}
    />
  );
}
```

Каждый ввод символа вызывает `onChange`, и мы обновляем состояние.

34. Что такое корневой элемент (root) в React-приложении?

Корневой элемент — это DOM-элемент, в который React «монтирует» всё приложение.

В HTML обычно:

html
```
<div id="root"></div>
```

В JS:

jsx
```
import ReactDOM from 'react-dom/client';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

Всё React-приложение живет внутри этого `div`.

35. Чем отличается `export default` от `export` в React?

Это не особенность React, а модулей JS, но часто встречается в React-коде.

- `export default` — модуль экспортирует **одну сущность по умолчанию**:

  js

  ```
  // Button.jsx
  export default function Button() {}
  ```

  Импорт:

  js

  ```
  import Button from './Button';
  ```

- **Именованный** `export`:

  js

  ```
  export const Button = () => {};
  export const Link = () => {};
  ```

  Импорт:

  js

  ```
  import { Button, Link } from './components';
  ```

Разница:

- `default` можно импортировать под любым именем.
- именованные экспорты требуют точного совпадения имени.

36. Как подключить картинку в React-компоненте?

Есть несколько способов, зависит от сборки, но типичные варианты:

1. **Импорт файла как модуля (для локальных картинок):**

   jsx

   ```
   import logo from './logo.png';

   function Header() {
     return <img src={logo} alt="Логотип" />;
   }
   ```

2. **Использовать путь из** `public` **(например, в Create React App):**
   - Положить картинку в `public/images/logo.png`.
   - Использовать:

     jsx

     ```
     function Header() {
       return <img src="/images/logo.png" alt="Логотип" />;
     }
     ```

3. **URL из интернета:**

   jsx

```
<img src="https://example.com/image.png" alt="Картинка" />
```